Mars Habitat Automation Platform

Distributed Automation for a Simulated Mars Habitat

MLPG

Mars Operations – Microservices-based IoT Platform

Outline

1. System Overview
2. Architecture and Containers
3. Technology Stack
4. User Stories
5. Conflict Management Logic
6. Frontend Mockups
7. Architecture Summary

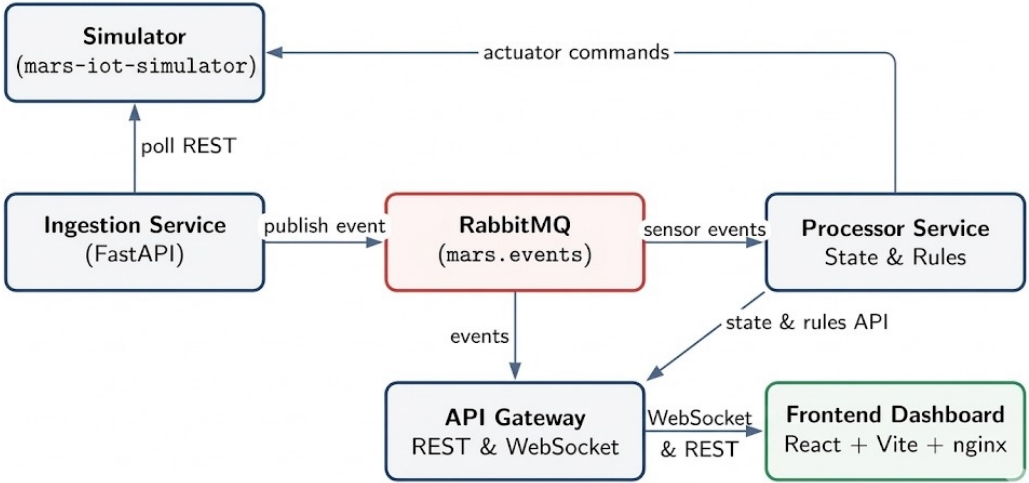
System Description

Goal

Provide an automation platform to **monitor** and **control** a simulated Mars habitat, starting from sensor data produced by the Mars IoT Simulator.

- **Unified data model**: all raw sensor and actuator signals are normalized into a single `UnifiedEvent` schema.
- **Event-driven architecture**: services exchange events through a RabbitMQ broker.
- **Automation rules engine**: IF–THEN rules evaluate incoming events and trigger actuator commands.
- **Manual control**: operators can set actuators ON/OFF from the dashboard.
- **Real-time dashboard**: a React SPA (Single Page Application) shows live readings, histories, rule status and conflicts.

High-level Data Flow



Docker Composition

- **simulator** (`port: 8080`): external Mars IoT Simulator, source of all sensor data and actuators.
- **rabbitmq** (`ports: 5672, 15672`): message broker, exchange `mars.events`, queues `processor.events`, `gateway.events`.
- **ingestion-service** (`port: 8001`): polls simulator REST (Representational State Transfer, a group of standard rules on how two programs should communicate, e.g. GET, POST, ...) endpoints and publishes normalized events.
- **processor-service** (`port: 8002`): state cache, rules engine, conflict arbitrator, SQLite persistence, actuator client.
- **api-gateway** (`port: 8003`): WebSocket endpoint `/ws`, REST proxies to processor and simulator.
- **frontend** (`port: 3000`): React SPA served via nginx (Engine-ex); proxies `/api` and `/ws` to the gateway.

Ingestion Service Responsibilities

Role in the platform

The Ingestion Service is the **entry point** for all sensor data coming from the Mars IoT Simulator.

- **Polling the simulator**
  - Periodically calls the simulator REST endpoints for each sensor (temperature, humidity, particulate, tank levels, ...).
- **Normalization into `UnifiedEvent`**
  - Converts heterogeneous payloads into the common `UnifiedEvent` schema.
- **Publishing to RabbitMQ**
  - Acts as a producer on the `mars.events` exchange.
  - It's fully stateless: in case of restart, it just reconnects and resumes polling the simulator.

Processor Service Responsibilities (1/2)

Core logic engine

The Processor Service is the **brain** of the platform: it tracks latest sensor state and decides when/how to act.

- **State management**
  - Consumes normalized events from RabbitMQ and updates in-memory `StateCache`.
  - Exposes REST endpoints for full/single-sensor state.
- **Rule evaluation**
  - Loads automation rules from SQLite and keeps a list of rules.
  - Evaluates conditions per incoming event and emits logical actuator commands.

Processor Service Responsibilities (2/2)

- **Conflict arbitration and safety**
  - Uses the `Arbitrator` to group commands per actuator in a short time window.
  - Detects ON/OFF conflicts and applies safe-state policy (default: `OFF`).
- **Persistence**
  - Stores rules in SQLite on a Docker volume so configuration survives restarts.

API Gateway and RabbitMQ

API Gateway

- Provides a **single entry point** for the frontend.
- Exposes `/ws` for live event streaming.
- Proxies `/api` calls to Processor (state/rules/conflicts) and simulator actuators.

RabbitMQ

- Central broker for sensor readings, actuator commands, and conflict alerts.
- Implements `mars.events` exchange with service queues.
- Decouples ingestion, processing, and streaming so services can scale/restart independently.

Frontend Dashboard Responsibilities

Operator-facing features

- Consumes WebSocket events and REST APIs from the gateway.
- Renders live sensor cards, actuator states, automation rules, and conflicts.
- Supports manual actuator control and rule actions (create/edit/toggle/delete).

Backend Technology Choices

Languages and frameworks

- **Python 3.11** for all internal services (ingestion, processor, API Gateway).
- **FastAPI** as the main web framework:
  - async-first, well aligned with message queues and WebSockets;

Supporting libraries (AIO versions nd Pydantic)

- **aio-pika**: asynchronous RabbitMQ client used by ingestion, processor and gateway to publish/consume events.
- **aiohttp**: async HTTP client to talk to the simulator and to proxy requests between services.
- **aiosqlite**: async SQLite driver used to store/retrieve rules without blocking the event loop.
- **Pydantic**: Ensures that the input data is correct.

Frontend and UX Technology Choices

Frontend stack

- **React 18** for building an interactive single-page dashboard capable of updating without refreshing the page.
- **Vite** as the build tool:
  - provides fast development pages

Serving and integration

- **nginx** serves the built static assets and proxies:
  - REST requests under `/api` to the API Gateway;
  - WebSocket connections under `/ws` to the same gateway.
- The frontend maintains its own in-memory state and histories, reacting to messages from the WebSocket and updates from REST.

Infrastructure Components

Message broker

- **RabbitMQ 3-management** is used as the central event bus.
- Defines the `mars.events` exchange and service-specific queues.

Database

- **SQLite** is chosen for persistence of automation rules:
  - embedded and lightweight; no external database service is required.
  - mounted on a Docker volume to survive container restarts.
- Only configuration data (rules) are persisted.

Complete List of User Stories (1/2)

1. **US01** – As an operator, I want to read sensors data, so that I can monitor the habitat.
2. **US02** – As an operator, I want to normalize sensors data, so that I can process them.
3. **US03** – As an operator, I want to create automation rules, so that I can automate the habitat.
4. **US04** – As an operator, I want to edit/delete existing automation rules, so that I can modify the behavior of the habitat.
5. **US05** – As an operator, I want to switch on/off automation rules, so that I can modify the behavior of the habitat.
6. **US06** – As an operator, I want to save automation rules, so that they persist after a restart.

Complete List of User Stories (2/2)

7. **US07** – As an operator, I want to interact manually with actuators, so that I can directly control the habitat.
8. **US08** – As an operator, I want to see the latest sensor readings, so that I can react immediately to critical changes in the habitat.
9. **US09** – As an operator, I want to check sensors status, so that I can detect malfunctions.
10. **US10** – As an operator, I want to see the evolution of sensors data, so that I can understand the behavior of the habitat.
11. **US11** – As the software, I want to show conflicts between automator rules, so that the operator can solve the issue.

User Stories (1/2): Monitoring and Data

- **US01 – US02**: **Read & Normalize** raw sensor data to enable continuous habitat monitoring.
- **US08**: **Live Readings** to react immediately to critical environmental changes.
- **US09**: **Sensor Status** to quickly detect anomalies and malfunctions.
- **US10**: **Data Evolution** to understand long-term habitat behavior through historical trends.

How we address them

Ingestion normalizes simulator data; the processor keeps the latest state; the dashboard shows live sensor cards, status badges and mini-charts.

User Stories (2/2): Automation, Control and Conflicts

- **US03 – US06**: **Rule Lifecycle Management** (create, edit, toggle, persist) to fully automate the habitat behavior.
- **US11**: **Rule Priority** to ensure critical automations take precedence.
- **US07**: **Manual Control** to directly interact with actuators.
- **US11**: **Conflict Detection** to automatically highlight overlapping rules so the operator can resolve them.

How we address them

Rules are stored and evaluated by the processor; the dashboard lets operators manage rules and control actuators; conflicts are strictly arbitrated and highlighted in the UI.

Conflict Resolution – Intuition

Context

Multiple automation rules can target the **same actuator** in the same evaluation cycle.

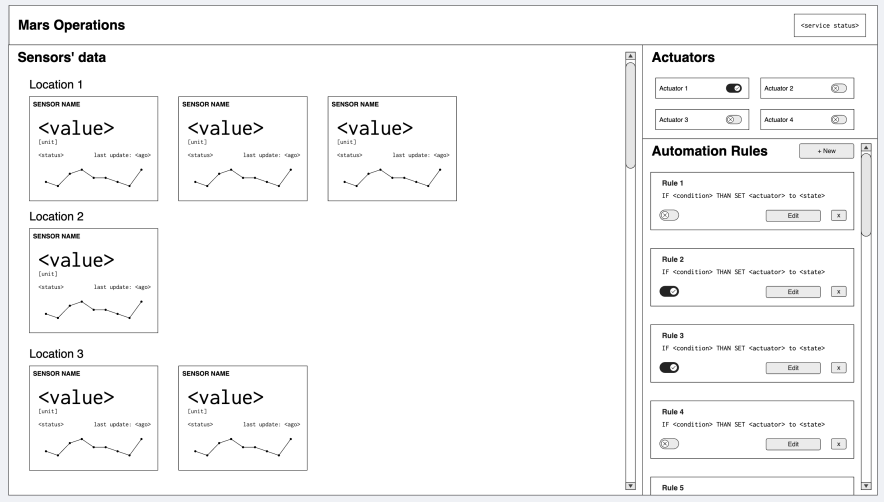
- To ensure safety, the system:
  - **groups** rule outcomes by actuator during a small time window and checks if there some rule that want to set an actuator `ON` and some rule that want to set an actuator `OFF`;
  - chooses a **safe default state** and highlights conflicts in the UI.
- This logic is entirely implemented in the processor-service `Arbitrator` component.

Conflict Handling (Backend)

How conflicts are handled

- When several rules target the same actuator in a short window, the **Arbitrator** groups commands.
- If all rules agree (all `ON` or all `OFF`), the actuator is set accordingly.
- If there was a previous warning for that actuator, it is cleared automatically.
- If at least one rule asks `ON` and at least one asks `OFF`, a **conflict** is detected.
- In conflict, the safe default is applied: actuator set to `OFF`.

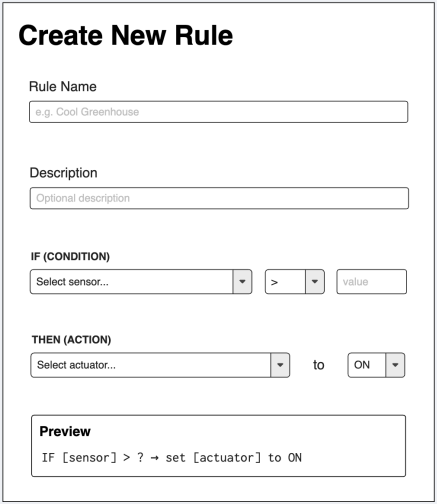
Dashboard Layout (Mockup)



Sensors Panel (Mockup)

- **Grouping by location**
  - Sensors are grouped under headers like *GREENHOUSE*, *ENTRANCE*, *HABITAT*.
  - Each group can display cards in grid or list view.
- **Sensor cards**
  - Show sensor name, location, last value and unit, status indicator and "last update" time.
  - Show a mini SVG chart showing recent history (min/max labels below the chart).
- **Multi-metric sensors**
  - For sensors such as `water_tank_level` or `air_quality_pm25`, the card behaves as a small carousel.
  - Each slide corresponds to a specific metric (e.g. *level pct*, *level liters*, *PM 2.5*).

New Automation Rule Layout (Mockup)



Automation Rules Panel (Mockup)

- **Rule cards**
  - Display the rule name and a readable IF–THEN sentence based on the internal condition.
  - Include a toggle switch to enable/disable the rule (`is_active`).
  - Provide *Edit* and *Delete* buttons; delete is confirmed with a second click.
- **Create / Edit modal**
  - Guides the user through selecting:
    - sensor (and metric, for multi-metric sensors);
    - comparison operator ($<, \leq, ==, >, \geq$);
    - threshold value;
    - target actuator and desired state.
  - Shows a live textual preview of the resulting rule.
- **Conflicts indication**
  - Rules involved in active conflicts are highlighted and display a warning icon with a tooltip.

Actuators Panel (Mockup)

- **Actuator tiles**
  - Each actuator (e.g. *cooling fan*, *habitat heater*) appears as a rectangular tile.
  - Color and icon indicate ON/OFF status at a glance.
- **Interaction**
  - Clicking a tile toggles its state and sends a POST request through the API Gateway to the simulator.
  - WebSocket updates keep tiles synchronized with actual simulator responses.

Technical Architecture Recap

- **Data ingestion**
  - Ingestion-Service polls the simulator, normalizes payloads into `UnifiedEvent`, and publishes them to RabbitMQ.
- **Processing and rules**
  - Processor-Service consumes events, updates the state cache, evaluates rules and arbitrates actuator commands.
  - Rules and their metadata are persisted in SQLite.
- **Exposure to operators**
  - API-Gateway exposes WebSocket and REST APIs to the frontend.
  - Frontend displays live data, histories, rule configuration, conflicts and actuators in a single dashboard.

Example Scenario: Automatic Cooling

1. Simulator reports a high `greenhouse_temperature` reading.
2. Ingestion-Service polls the value, builds a `UnifiedEvent` and publishes it to RabbitMQ.
3. Processor-Service consumes the event; the rule engine finds a rule such as:
   IF greenhouse temperature > threshold THEN set cooling fan to ON.
4. The rule triggers and emits a command to the arbitrator; if no conflicting rule asks for OFF, the arbitrator sets the cooling fan to ON.
5. The actuator command is sent to the simulator and an `actuator_command` event is published.
6. Gateway forwards events via WebSocket; dashboard updates sensors, rule status, and cooling fan tile.

Thank You!

MLPG

Mars Operations – Microservices-based IoT Platform